ChatGPT 5.2

Share

how to apply taint on node

Applying a taint to a node is straightforward once you know the syntax.

At a high level, a **taint = node repellent**.
It tells Kubernetes: *"Do not schedule pods here unless they tolerate this taint."*

✅ Basic Command

Bash

```bash
kubectl taint nodes <node-name> <key>=<value>:<effect>
```

✅ Example

Bash

```bash
kubectl taint nodes worker-node1 dedicated=database:NoSchedule
```

Meaning:

- **key** → `dedicated`
- **value** → `database`
- **effect** → `NoSchedule`

👉 Result: Pods **won't be scheduled** on `worker-node1`
unless they have a matching **toleration**.

✅ Effects You Can Use

1️⃣ NoSchedule

Hard block. Scheduler avoids node.

Bash

```
kubectl taint nodes node1 env=prod:NoSchedule
```

2️⃣ PreferNoSchedule

Soft block. Scheduler tries to avoid.

```bash
kubectl taint nodes node1 env=prod:PreferNoSchedule
```

3️⃣ NoExecute

Stronger:

✔ Blocks new pods
✔ Evicts existing pods (without toleration)

```bash
kubectl taint nodes node1 env=prod:NoExecute
```

✅ Verify Taint

```bash
kubectl describe node <node-name>
```

Look for:

```Code
Taints: dedicated=database:NoSchedule
```

✅ Remove Taint

Add a trailing minus `-`

```bash
```

```
kubectl taint nodes worker-node1 dedicated=database:NoSchedule-
```

🧠 Practical Real-World Uses

People taint nodes for:

✔ Dedicated workloads (DB / GPU / Monitoring)
✔ Special hardware nodes
✔ Prevent accidental scheduling
✔ Temporary maintenance isolation

Example:

Bash

```bash
kubectl taint nodes gpu-node accelerator=true:NoSchedule
```

Only ML pods with tolerations land there.

If you'd like, next logical step:

👉 How pods tolerate taints (tolerations YAML examples)

This is where interviews love to dig deeper 😏

created a cluser with kind .
can i apply taint while creating cluster ?
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 30001 # KIND Control Plane Container
    hostPort: 30001 # Laptop:30001
  - containerPort: 443
    hostPort: 30443
  - containerPort: 80
    hostPort: 30080
- role: worker

```
- role: worker
networking:
  disableDefaultCNI: true
  podSubnet: 192.168.0.0/16
```

Short answer: **kind does NOT support taints directly in the cluster config YAML.**

There is **no native** `taints:` **field** in:

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
```

So if you try to declare taints there → kind simply ignores it.

✅ Correct Way with kind

You taint **after the cluster is created**, using normal Kubernetes commands.

Example:

```bash
kubectl taint nodes kind-worker dedicated=test:NoSchedule
```

Works exactly like any other cluster.

✅ But… Can We Automate It During Creation?

Yes 😏
We cheat a little using **kubeadm patches**.

Kind internally uses **kubeadm**, and kubeadm *does* support taints.

✅ Taint Node at Cluster Creation (Advanced / Real Answer)

You inject taints via:

YAML

```yaml
kubeadmConfigPatches:
```

Example 👇

YAML

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4

nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      taints:
      - key: "node-role.kubernetes.io/control-plane"
        effect: "NoSchedule"

- role: worker
  kubeadmConfigPatches:
  - |
    kind: JoinConfiguration
    nodeRegistration:
      taints:
      - key: "dedicated"
        value: "apps"
        effect: "NoSchedule"

- role: worker
```

✅ What Happens Here?

✔ kubeadm applies taints when node registers

✔ No need for manual `kubectl taint`

✔ Very useful for labs / demos / exams

⚠️ Important Interview Insight

Most people don't know this:

👉 kind config ≠ Kubernetes API

Kind is just a wrapper around kubeadm.

So:

✔ Want kubelet-level settings → use patches
✔ Want scheduling behavior → taints / labels via patches

✅ Verify

Bash

```bash
kubectl describe node kind-worker
```

You'll see:

Code

```
Taints: dedicated=apps:NoSchedule
```

If you want, next fun trick:

👉 **Auto-label nodes in kind during creation**
👉 **Create dedicated GPU / infra nodes in kind** 😉